algorithms

MDPI

Article

Mxplainer: Explain and Learn Insights by Imitating Mahjong Agents

Lingfeng Li, Yunlong Lu, Yongyi Wang, Qifan Zheng and Wenxin Li *

School of Computer Science, Peking University, Beijing 100871, China; lingfengli@stu.pku.edu.cn (L.L.); luyunlong@pku.edu.cn (Y.L.); wangyongyi@pku.edu.cn (Y.W.); zqf1998@stu.pku.edu.cn (Q.Z.)
* Correspondence: lwx@pku.edu.cn

Abstract

People need to internalize the skills of AI agents to improve their own capabilities. Our paper focuses on Mahjong, a multiplayer game involving imperfect information and requiring effective long-term decision-making amidst randomness and hidden information. Through the efforts of AI researchers, several impressive Mahjong AI agents have already achieved performance levels comparable to those of professional human players; however, these agents are often treated as black boxes from which few insights can be gleaned. This paper introduces Mxplainer, a parameterized search algorithm that can be converted into an equivalent neural network to learn the parameters of black-box agents. Experiments on both human and AI agents demonstrate that Mxplainer achieves a top-three action prediction accuracy of over 92% and 90%, respectively, while providing faithful and interpretable approximations that outperform decision-tree methods (34.8% top-three accuracy). This enables Mxplainer to deliver both strategy-level insights into agent characteristics and actionable, step-by-step explanations for individual decisions.

Keywords: explainable AI; card games; machine learning

Academic Editor: Frank Werner

Received: 27 August 2025
Revised: 11 November 2025
Accepted: 22 November 2025
Published: 24 November 2025

Citation: Li, L.; Lu, Y.; Wang, Y.; Zheng, Q.; Li, W. Mxplainer: Explain and Learn Insights by Imitating Mahjong Agents. *Algorithms* **2025**, *18*, 738. https://doi.org/10.3390/a18120738

1. Introduction

Games play a pivotal role in the field of AI, offering unique challenges to the research community and serving as fertile ground for the development of novel AI algorithms. Board games, such as Go [1] and chess [2], provide ideal settings for perfect-information scenarios, where all agents are fully aware of the environment states. Card games, like heads-up no-limit hold'em (HUNL) [3,4], Doudizhu [5,6], and Mahjong [7], present different dynamics with their imperfect-information nature, where agents must infer and cope with hidden information from states. Video games, such as Starcraft [8], Minecraft [9], and Honor of Kings [10], push AI algorithms to process and extract crucial features from a multitude of signals amidst noise.

Conversely, the advancement of AI algorithms also incites new enthusiasm in games. The work of AlphaGo [11] in 2016 has had a long-lasting impact on the Go community. It revolutionized the play style of Go, a game with millennia of history, and changed the perspectives of world champions on this game [12]. Teaching tools based on AlphaGo [13] have become invaluable resources for newcomers while empowering professional players to set new records [14].

Mahjong, a worldwide popular game with unique characteristics, has gained traction in the AI research community as a new testbed. It brings its own flavor as a multiplayer imperfect-information environment. First, there is no rank of individual tiles in Mahjong;

all tiles are equal in their role within the game. A game of Mahjong is not won by beating other players in card ranks but by being the first to reach a winning pattern. Therefore, state evaluation is more challenging for Mahjong players, as they need to assess the similarity and distance between game states and the closest game goals. Second, the game objective in Mahjong is to become the first to complete one of many possible winning patterns. The optimal goal can frequently change when players draw new tiles during gameplay. In fact, Mahjong's core difficulty lies in selecting the most effective goal among numerous possibilities. Players must evaluate their goals and make decisions upon drawing each tile and reacting to other players' discarded tiles. This decision-making process often involves situations where multiple goals are of similar distance, requiring trade-offs that distinguish play styles and reveal a player's level of expertise.

Several strong agents have already been developed for different variants of Mahjong rules [7,15,16]. However, as illustrated in Figure 1, without the use of explainable AI methods [17,18], people can only observe the agents' actions without understanding how the game states are evaluated or which game goals are preferred that lead to those actions.

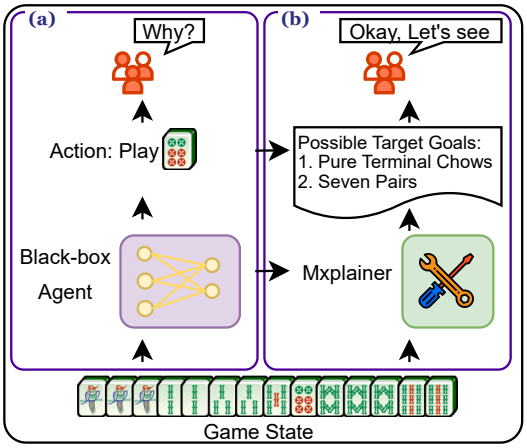

Figure 1. Comparison between current situation of black-box Mahjong agents and Mahjong agents with Mxplainer. (**a**) Raw action output without explanations. (**b**) Mxplainer explains possible reasons behind actions.

In Explainable AI (XAI) [17], black-box models typically refer to neural networks that lack inherent transparency and thus rely on post hoc explanation tools for interpretability [18]. Existing XAI techniques, such as LIME [19] and LMUT [20], struggle to provide both high-accuracy policy approximation and human-intelligible explanations in domains with immense state spaces and non-Markovian characteristics, a gap that becomes critical for analyzing strategic agents in games like Mahjong.

In this paper, we present Mxplainer (**M** ahjong E**xplainer**), a parameterized classical agent framework designed to serve as an analytical tool for explaining the decision-making process of black-box Mahjong agents, as shown in Figure 1b. Specifically, we have developed a parameterized framework that forms the basis for a family of search-based Mahjong agents. This framework is then translated into an equivalent neural network model, which can be trained using gradient descent to mimic any black-box Mahjong agent. Finally, the learned parameters are used to populate the parameterized search-based agents. We consider these classical agents to be inherently explainable because each calculation and decision step within them is comprehensible to human experts. This enables detailed interpretation and analysis of the decision-making processes and characteristics of the original black-box agents.

Through a series of experiments on game data from both AI agents and human players, we demonstrate that the learned parameters effectively reflect the decision processes of

agents, including their preferred game goals and tiles to play. Our research also shows that by delving into the framework components, we can interpret the decision-making process behind the actions of black-box agents.

This paper pioneers research on analyzing Mahjong agents by presenting Mxplainer, a framework to explain black-box decision-making agents using search-based algorithms. Mxplainer allows AI researchers to profile and compare both AI agents and human players effectively. Additionally, we propose a method to convert any parameterized classical agent into a neural agent for automatic parameter tuning. Beyond traditional approaches like decision trees, our work explores the conversion from neural agents to classical agents.

The rest of this paper is organized as follows: We first review the related literature. Next, we introduce the rules of Mahjong and the specific variant used in our study. Then, we provide a detailed explanation of the components of our approach. Following this, we present a series of experiments demonstrating the effectiveness of our method in approximating and capturing the characteristics of different Mahjong agents. Finally, we discuss the implications of our work and conclude with future research directions.

2. Related Works

Explainable AI (XAI) [17] is a research domain dedicated to developing techniques for interpreting AI models for humans. This field encompasses several categories: classification models, generative AI, and decision-making agents. A specific subfield within this domain is Explainable Reinforcement Learning (XRL) [21], which focuses on explaining the behavior of decision-making agents. Explanations in XRL can be classified into two main categories: global and local.

Global explanations provide a high-level perspective on the characteristics and overall strategies of black-box agents, answering questions such as how an agent's strategy differs from others. Local explanations, on the other hand, focus on the detailed decision-making processes of agents, elucidating why an agent selects action A over B under specific scenarios.

XRL methods can be classified into intrinsic and post hoc methods. Intrinsic methods directly generate explanations from the original black-box models, while post hoc methods rely on additional models to explain existing ones. Imitation learning (IL) [22,23] is a family of post hoc techniques that approximate a target policy. LMUT [20] constructs a U-tree with linear models as leaf nodes and approximates target policies through a node-splitting algorithm and gradient descent. Q-BSP [24] uses a batched Q-value to partition nodes and generate trees efficiently. EFS [25] employs an ensemble of linear models with non-linear features generated by genetic programming to approximate target policies. These methods have been tested and excelled in environments such as CartPole and MountainCar, and others have excelled in the Gym [26]. However, they rely on properly sampled state–action pairs to generate policies and may not be robust to out-of-distribution (OOD) states, which is particularly crucial for Mahjong with its high-dimensional state space and imperfect information.

PIRL [27] distinguishes itself among IL methods by introducing parameterized policy templates using its proposed policy programming language. It approximates the target π through fitting parameters with Bayesian Optimization in Markov games, achieving high performance in the TORCS car racing game. Compared to TORCS, Mahjong has far more complex state features and requires encoding action histories within states to obtain the Markov property. Additionally, Mahjong agents must make multi-step decisions from game goal selection to tile picking. Similarly to PIRL, we define a parameterized search-based framework and optimize parameters using batched gradient descent to address these challenges.

3. Mahjong: The Game

Mahjong is a four-player imperfect information tile-based tabletop game. The complexity of imperfect-information games can be measured by information sets, which are game states that players cannot distinguish from their own observations. The average size of information sets in Mahjong is around 10^{48}, making it a much more complex game to solve than Heads-Up Texas Hold'em [28], for which its average information set size is around 10^3. This immense complexity arises from the vast number of possible tile distributions, hidden player hands, and the long sequence of interdependent decisions involving drawing, discarding, and claiming tiles over multiple rounds. To facilitate the readability of this paper, we highlight terminologies used in Mahjong with **bold texts**, and we distinguish scoring patterns (**fans**) by *italicized texts*.

In Mahjong, there are a maximum of 144 tiles, as shown in Figure 2A. Despite its plethora of rule variants, Mahjong's general rules are the same. On a broad level, Mahjong is a pattern-matching game. Each player begins with 13 tiles only observable to themselves, and they take turns to draw and discard 1 tile until one completes a game goal with a 14th tile. The general pattern of 14 tiles is four **melds** and a pair, as shown in Figure 2C. A meld can take the form of **Chow**, **Pung**, and **Kong**, as shown in Figure 2B. Apart from drawing all the tiles by themselves, players can take the tile just discarded by another player instead of drawing one to form a **meld**, called **melding**, or declare a win.

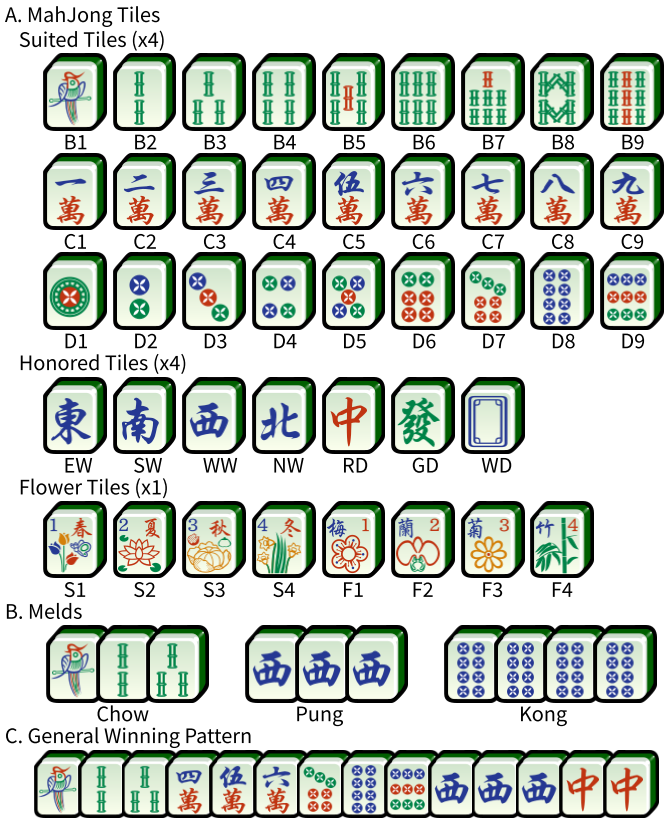

Figure 2. Basics of Mahjong. (**A**) All the Mahjong tiles. There are four identical copies for suited tiles and honored tiles, and one copy for each flower tile. (**B**) Examples of Chow, Pung, and Kong. Note that only suited tiles are available for Chow. (**C**) Example of the general winning pattern.

Official International Mahjong

Official International Mahjong stipulates Mahjong Competition Rules (MCRs) to enhance the game's complexity and competitiveness and weaken its gambling nature. It specifies 80 scoring patterns with different points, called "**fan**", ranging from 1 to 88 points.

In addition to the winning patterns of four melds and a pair, players must have at least eight points by matching multiple scoring patterns to declare a win.

Specific rules and requirements for each pattern can be found in [29]. Of 81 **fans**, 56 are highly valued and called **major fans** since most winning hands usually consist of at least one. The standard strategy of MCR players is to make game plans by identifying several **major fans** closest to their initial set of tiles. Then, depending on their incoming tiles, they gradually select one of them as the terminal goal and strive to collect all remaining tiles before others do. The exact rules of MCR are detailed in *Official International Mahjong: A New Playground for AI Research* [28].

4. Methods

We first introduce the general concepts of Mxplainer and then present the details of the implementations in the following subsections. To facilitate readability, we use uppercase symbols to indicate concepts, and lowercase symbols with subscripts indicate instances of concepts.

Figure 3 presents the concept overview of Mxplainer. We assume that the search-based nature of F is explainable to experts in the domain, such as Mahjong players and researchers. Within F, there are parameters Θ that control the behaviors of F. To explain a specific target agent Ψ, we would like the behaviors of F to approximate those of Ψ as closely as possible. In order to automate and speed up the approximation process, we convert a part of F that contains Θ into an equivalent neural network representation, and we leverage supervised learning to achieve the goals.

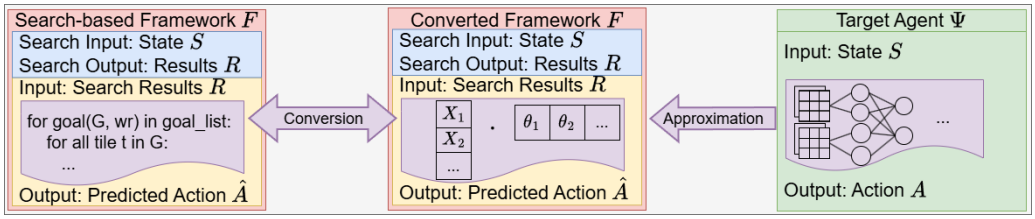

Figure 3. An overview of Mxplainer. Search-based framework F is a manually engineered, domain-specific, parameterized template. A component of F can be converted into an equivalent network for supervised learning, where parameters of F serve as neurons. Target agents Ψ are black-box agents to be analyzed, and they can be approximated by the converted F.

F consists of Search Component SC and Calculation Component CC, denoted as $F = SC|CC$. SC searches and proposes valid goals in a fixed order. Next, CC takes groups of manually defined parameters Θ, each of which carries meanings and is explainable to experts, and makes decisions based on the search results from SC, as shown in Figure 4.

CC can be converted into an equivalent neural network N, for which its neurons are semantically equivalent to Θ. $SC|N$ can approximate any target agents Ψ by fitting Ψ's state–action pairs. Since Θ is the same for both CC and N, learned $\hat{\Theta}$ can be put back into CC and explains actions locally through step-by-step deductions of $SC|CC$. Moreover, by normalizing and studying Θ, Mxplainer is able to compare and analyze agents' strategies and characteristics.

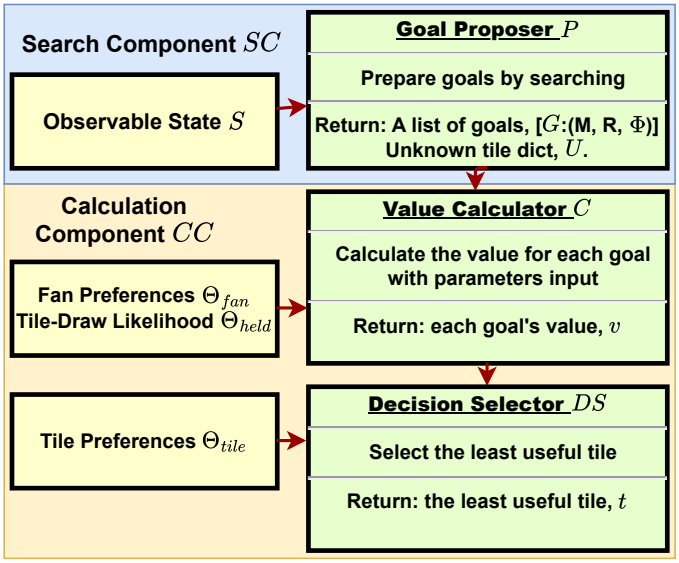

Figure 4. Components of framework F. Search Component SC uses Goal Proposer P for goal search. Calculation Component CC consists of Value Calculator C and Decision Selector DS, and it is responsible for action calculations. The least useful tile t can be considered as the tile that appears least frequently among the most probable goals. Consequently, removing this tile has the least impact on the expected value.

The construction of F and the design of parameters Θ are problem-related, as they reflect researchers' and players' attention to the game's characteristics and the game agents' strategies. The conversion from CC to N and the approximation through supervised learning is problem-agnostic and can be automated. In the Mxplainer framework, the Search Component (SC) of F is fixed and shared across all agents, while the behavior of the Calculation Component (CC) and its neural equivalent N is determined by the parameters Θ, which are tuned to mimic different target agents Ψ.

4.1. Parameters Θ of Framework F

For Θ, we manually craft three groups of parameters to model people's general understanding of Mahjong: Θ_{tile}, Θ_{fan}, and Θ_{held}.

Θ_{tile} is used to break ties between tiles, and different players may have different preferences for tiles.

Θ_{fan} is designed to break ties between goals when multiple goals are equally distant from the current hand. There are more than billions of possible goals in Mahjong, but each goal consists of **fans**. Thus, we use the compound of preferences of **fans** to sort goals. We hypothesize that there exists a natural order of difficulty between **fans**, which implies that some are more likely to achieve than others [29]. However, such an order is impossible to obtain unless there is an oracle that always gives the best action. Additionally, players break ties based on their inclinations towards **fans**. Since the natural difficulty order and players' inclinations are hard to decouple, we use Θ_{fan} to represent their products. Consequently, Θ_{fan} between different **fans** for a single player cannot be compared directly, but Θ_{fan} for the same **fans** between different players reflect their comparative preferences.

Θ_{held} denotes the linear weights of a heuristic function that approximates the probability that a tile is held by other players. The linear function takes features from local game states, including the number of total unshown tiles, the length of game steps, and the unshown tile counts of the neighboring tiles. Unshown tiles are those not visible to the agent, encompassing tiles in the wall and in opponents' hands. The agent maintains a probability distribution over these tiles, modeled as a dictionary U, where each key is a tile

type and its value is the estimated number of remaining copies. The components and the usage of Θ_{held} will be discussed in detail in the following sections.

4.2. Search Component SC of Framework F

In Mahjong, **Redundant Tiles**, R, refer to the tiles in a player's hand that are considered useless for achieving their game goals. In contrast, **Missing Tiles**, M, are the ones that players need to acquire in future rounds to complete their objectives. Following this, the **Shanten Distance** is defined as $D = |M|$, which conveys the distance between the current hand and a selected goal.

Here, we define a game goal as $G = (M, R, \Phi)$, since when both M and R are fixed, a game goal is set, and its corresponding **fans** Φ can be calculated. Each tile $t \in M$ additionally has two indicators, i_p and i_c, which determine if it can be acquired through melding from other players. i_p is true if the player already owns two tiles in the **Pung**, and the same happens to i_c and **Chow**. Thus, $\forall t \in M$, it has (t, i_p, i_c).

Through dynamic programming, we can efficiently propose different combinations of tiles and test if they satisfy MCR's winning condition. We can search all possible goals, but not all goals are reasonable to be considered as candidates. In practice, only up to 64 goals, G, are returned in ascending **Shanten Distances** D, since in most cases, only the closest goals are important for decisions, and they are usually less than two dozen.

However, only the presence of goals is not enough. MCR players also need to consider other observable game information to jointly evaluate the value of each goal G, such as other players' discard history and unshown tiles. For our framework F, we only consider unshown tiles U, a dictionary keeping track of the number of each tile that is not shown through observable information.

Thus, the Goal Proposer P accepts game state information S and outputs $([G], U)$ such that $0 < |[G]| \leq 64$, as shown in Figure 4A. In most cases, goals, G, in $[G]$ can be split into several groups. Different groups contain different **fans** Φ and represent different paths to win, while the goals within each group have different tile combinations for the same **fans** Φ.

4.3. Calculation Component CC of Framework F

Calculation Component CC of Framework F consists of Value Calculator C and Decision Selector DS. CC contains three groups of tunable parameters Θ_{fan}, Θ_{held}, and Θ_{tile} that control the behavior of Value Calculator C and Decision Selector DS. In all, these three groups of parameters, Θ_{fan}, Θ_{held}, and Θ_{tile}, are similar to parameters of neural networks; they are the key factors that control the behaviors of agents derived from framework F.

The Value Calculator C takes the results of the Goal Proposer P, $([G], U)$ as input and estimates the value for each goal G. The detailed algorithm of C is shown in Algorithm 1. Simply put, C multiplies the probability of successfully collecting each **missing tile** of a proposed goal as its estimated value.

The value estimation in the Value Calculator C is governed by two parameter groups: Θ_{fan} and Θ_{held}. The probability of acquiring a required tile t (t denotes a generic tile, and $t \in X$ indicates that tile t belongs to a set X) is decomposed into two components: drawing it from the wall or **melding** it from another player's discard.

The probability of drawing tile t is modeled as follows:

$$P_{draw}(t) = U[t]/\Sigma(U) \cdot Z \cdot \Theta_{held}$$

This formulation reflects the assumption that the likelihood of drawing t is proportional to the number of its remaining unshown copies $U[t]$ and inversely proportional to the total number of unshown tiles $\Sigma(U)$. Furthermore, the term $Z \cdot \Theta_{held}$ accounts for the

effect of other players potentially holding the tile, where Z denotes a feature vector derived from local game state attributes, and Θ_{held} serves as a set of linear weights:

$$Z : \{\Sigma(U), \frac{1}{\Sigma(U)}, 1 - \frac{1}{\Sigma(U)}, L, \frac{1}{L}, 1 - \frac{1}{L}, U[t-2] : U[t+2], bias\}$$

Algorithm 1 Value Estimation for A Single Goal

Input: Goal G: $<$ **missing tiles** M:$< t$, **Chow** indicator i_c, **Pung** indicator $i_p>$, **fan** list $F >$, unshown dict U, game length L
Parameter: $\Theta_{fan}, \Theta_{held}$
Output: Estimated value for goal G

Initialize value $v \leftarrow 100$
for all missing tile $m \in M$ **do**
  // Construct local tile feature x_m from game length L,
  // and remaining adjacent tile count U.
  $x_m \leftarrow U, L$
  // Calculate probability of drawing m and others discarding m
  $p_{draw} \leftarrow U[m]/sum(U) * \Theta_{held} * x_m$
  $p_{discard} \leftarrow U[m]/sum(U)$
  $source \leftarrow 0$
  if i_p **then**
    $source \leftarrow 3$ // one can pung from all others
  else if i_c **then**
    $source \leftarrow 1$ // one can only **chow** from the one left
  end if
  $p_{meld} \leftarrow p_{discard} * source$
  $v \leftarrow v \times (p_{draw} + p_{meld})$
end for
Total **fan** weight $fw \leftarrow \sum \Theta_{fan}[f]$ for **fan** $f \in F$
$v \leftarrow v \times fw$
return wr

Here, L denotes the current game length, and $U[t-2] : U[t+2]$ represents the unshown counts of tiles adjacent to t. Including a bias term, Θ_{held} contains 12 learnable weights in total. In contrast, the probability of **melding** tile t from another player's discard is modeled using a uniform distribution, $P_{meld}(t) = U[t]/\Sigma(U)$. We examine alternative probability models for P_{draw} and P_{meld} in Section 6.2, and we further discuss the rationale behind our modeling choices in Section 7.1.

The Decision Selector DS collects results from the Value Calculator C and calculates the final action based on each goal's estimated value. The detailed algorithm for S is shown in Algorithm 2. The goal of S is to efficiently select the tile to discard with the most negligible impact on the overall value. Heuristically, the required tiles of goals with higher values are more important than those with lower ones. Conversely, the **redundant tiles** of such goals are more worthless since their existence actually hinders the goals' completion. Thus, each tile's worthless degree can be computed by accumulating the value of goals that regard it as a **redundant tile**, and the tile with the highest worthless degree has the most negligible impact on the overall value. Decision Selector DS accepts Θ_{tile} as parameters, which are used similarly as Θ_{fan} to break ties between tiles. For action predictions, such as **Chow** or **Pung**, F records values computed by C, assuming those actions are taken, and F selects the action with the highest value.

Algorithm 2 Discarding Tile Selection

Input: List of goals and their values $L = [(\text{Goal } G, \text{value } v)]$, Hand tiles H
Parameter: Θ_{tile}
Output: Tile to discard

Initialize tile values $d \leftarrow$ Dict $\{\text{tile } t : 0\}$
for all $(G, v) \in L$ **do**
  for all tile $t \in G$ **do**
    $d[t] \leftarrow d[t] + v$ // **redundant tiles**
  end for
end for
for all tile $t \in d$ **do**
  $d[t] \leftarrow d[t] \times \Theta_{tile}[t]$
end for
return $\arg\max_{t \in H} d[t]$

4.4. Differentiable Network N of Mxplainer

The search-based Framework F is a parameterized search-based agent template, and its parameter Θ needs to be tuned to approximate any target agents' behaviors. Luckily, Calculator C and Decision Selector DS only contain fixed-limit loops, with each iteration independent from others and if–else statements whose outcomes can be pre-computed in advance. Thus, C and S can be converted into an equivalent neural network N for parallel computation, and the parameters Θ can be optimized through batched gradient descent. The rules for the conversion can be found in Appendix B.

$$\mathbb{I} = \begin{cases} 1 & \text{if it is real data} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

To distinguish between padding and actual values, we define a value indicator as Equation (1) to facilitate parallel computations. Then, Algorithm 1 and Algorithm 2 can be easily re-written as Listing 1 and Listing 2. The inputs to the Listings are as follows:

- $F \in \mathbb{R}^{80}$, $U \in \mathbb{N}^{34}$, $H \in \mathbb{N}^{64\times34}$, $X \in \mathbb{R}^{34\times12}$: Vectorized F, U, H, and x_m from Algorithm 1 and Algorithm 2.
- $MT : (\mathbb{I}, M) \in \mathbb{R}^{34\times2}$: **Missing tiles** M with indicators.
- $BM \in \mathbb{N}^{34\times3}$: One-hot branching mask from i_c and i_p.
- $V \in \mathbb{R}^{64\times1}$: The computed value from Listing 1.
- w_held, w_fan, w_tile: Θ_{held}, Θ_{fan}, and Θ_{tile}.

Listing 1. Paralleled Algorithm 1 in PyTorch 2.3.1

```
def calc_winrate(F, MT, BM, X, U, w_held, w_fan):
    v = 100
    p_uniform = MT[:, 1]* U/torch.sum(U)
    p_draw = p_uniform*X*w_held
    p_discard = p_uniform
    # encode chi, peng, no-op coeff
    cp_cf = torch.tensor([3, 1, 0])
    # compute tile-wise cp_coeff
    cp_cf = cp_cf * BM
    cp_cf = torch.sum(cp_cf, dim=1)
    p_meld = p_discard*cp_cf
    p_tile = p_darw+p_meld
    # use indicator to set paddings 0
    masked_tile = MT[:, 0]*p_tile
    # convert 0's to 1 for product op
    masked_tile += 1-MT[:, 0]
    v *= torch.prod(masked_tile, dim=0)
    #calculate preference for fan
    p_fan = F*w_fan
    return v*p_fan
```

Listing 2. Paralleled Algorithm 2 in PyTorch

```
def sel_tile(H, V, w_tile):
    tile_v = H*V
    tile_v = torch.sum(tile_v, dim=0)
    tile_pref = tile_v*w_tile
    return torch.argmax(tile_pref)
```

The Resulting Network N and the Training Objective

The sizes and meanings of the network N's outputs and three groups of parameters are reported in Table 1. Action Kong is not explicitly encoded in the action space. Instead, it is logically implied and automatically executed by the Mxplainer agent under the following conditions:

- **Converting a Melded Pung:** The agent has an existing, exposed **Pung** of tile t and tries to discard the fourth t. The agent then executes **Kong** automatically.
- **Forming a Concealed Kong:** The agent has three identical tiles t concealed in its hand. **Kong** is triggered when it draws the fourth t and tries to discard it.
- **Forming a Kong:** The agent has three identical tiles t concealed in its hand. **Kong** is triggered automatically, when another player plays tile t, and the agent tries to **Pung** and discard t at the same time.

Table 1. Sizes and meanings of neural network N's components.

Entry	Size	Meaning
Overall Output	39	Combined output size
Output (Action)	5	5 Actions: Pass, 3 types of **Chow** , **Pung**
Param Θ_{fan}	80	Preference for all 80 **fans**
Params Θ_{held}	12	Linear features weights w/bias
Param Θ_{tile}	34	Preference for all 34 tiles

In the framework's decision process, declaring **Kong** in these scenarios is always the optimal action, as it increases the hand's value without altering the strategic plan. This design formally decomposes the policy into a piecewise-linear component for high-level actions and a domain-logic component for specialized decisions like discards and Kongs. The resulting composition can be viewed as building a complex policy from simpler, interpretable sub-policies, a concept with rigorous footing in optimization theory [30–32].

The learning objectives depend on the form of target agents Ψ and the problem context. Since we are modeling the selection of actions as classification problems, we use cross-entropy (CE) loss between the output of N and the label Y. The label can be soft or hard depending on whether the target agent gives probability distributions. Since we cannot access action distributions from Botzone's game log, we use actions as hard ground-truth labels for all target agents. Additionally, an L2-regularization term of $(\Theta_{fan} - abs(\Theta_{fan}))^2$ is added to penalize negative values of **fan** preferences to make the heuristic parameters more reasonable.

We employ supervised learning for this imitation task as it provides a direct and stable optimization objective to achieve high behavioral fidelity, which is a prerequisite for generating faithful explanations.

After training, the learned parameters Θ can be directly filled back into CC. Since CC is equivalent to N, $SC|CC(\Theta)$ inherits the characteristics of $SC|N(\Theta)$, approximating the target agent Ψ. By analyzing the parameterized agent $SC|CC(\Theta)$, we can study the parameters to learn the comparative characteristics between agents and gain insights into the deduction process of Ψ through the step-by-step algorithms of Framework $SC|CC$.

Since only data pairs are required from the target agents, we can use Mxplainer on both target AI agents and human players.

5. Experiments

We conducted a series of experiments to evaluate the effectiveness of Mxplainer in generating interpretable models and analyze their explainability. Three different target agents are used in these experiments. The first agent ψ_1 is a search-based MCR agent with manually specified characteristics as a baseline. Specifically, this agent only considers the **fan** of *Seven Pairs* to choose, and all its actions are designed to reach this target as efficiently as possible. When multiple tiles are equally good to discard, a fixed order is predefined to break the equality. The second agent ψ_2 is the strongest MCR AI from an online game AI platform, Botzone [33]. The third agent ψ_3 is a human MCR player from an online Mahjong platform, MahjongSoft.

Around 8000 and 50,000 games of self-play data are generated for the two AI agents. Around 34,000 games of publicly available data are collected from the game website for the single human player over more than 3 years. The datasets for each agent were split into training, validation, and test sets on a per-game basis to prevent data leakage and ensure a robust evaluation. Through supervised learning on these datasets, three sets of parameters $\theta_{1,2,3}$ are learned and filled back in the search-based framework F as-is, creating interpretable white-box agents $\hat{\psi}_{1,2,3}$ with similar behavior to the target agents $\psi_{1,2,3}$. The weights for each agent are selected with the highest validation accuracy from three runs. To make weights comparable across different agents, reported weights are normalized with $100 \times (\Theta - \Theta_{min})/\Sigma(\Theta - \Theta_{min})$.

Since it is common in MCR where multiple **redundant tiles** can be discarded in any order, it is difficult to achieve high top-1 accuracy on the validation sets, especially for ψ_2 and ψ_3, which have no preference on the order of tiles to discard. As a reference of the similarity, $\hat{\psi}_{1,2,3}$ achieves the top-three accuracy of 97.15%, 93.47%, 90.12% on the data of $\psi_{1,2,3}$ (All the accuracy figures presented in this paper only take into account states ≥ 2 valid actions, which account for less than 30% of all states. The overall accuracy can be roughly calculated as $70 + 0.3 \times acc$, where acc represents the reported accuracy.)

5.1. Correlation Between Behaviors and Parameters

In the search-based framework F, the parameters, Θ, are designed to be strategy-related features that should take different values for different target agents. Here, we analyze the correlation between the learned parameters and target agents' behavior to prove that Mxplainer can extract high-level characteristics of agents and explain their behavior.

5.1.1. Preference of **Fans** to Choose

Θ_{fan} stands for the relative preference of agents to win by choosing each **fan**. For the baseline target ψ_1, which only chooses the **fan** of *Seven Pairs*, the top values of $\theta_{1,fan}$ learned are shown in Table 2B. We also include $\theta_{2,fan}$ values for comparisons, demonstrating the differences in learned weights between specialized *Seven Pairs* agent and other agents. It can be seen that the weight for *Seven Pairs* is much higher than those of other **fans**, showing the strong preference of $\hat{\psi}_1$ to this **fan**. The **fans** with the second and third largest weights are patterns similar to *Seven Pairs*, indicating that $\hat{\psi}_1$ also tends to approach them during the gameplay based on its learned action choices, though it eventually ends with *Seven Pairs* for 100% of the games. For targets ψ_2 and ψ_3 with unknown preferences of **fans**, we count the frequency of occurrences of each **major fan** in their winning hands as an implication of their preferences and compare these two agents on both the frequency and the learned weights of each **fan**. Table 2C shows all the **major fans** with a difference of

1% in frequency. Except for the last row, the data shows a significant positive correlation between the preference of target agents ψ and the learned θ_{fan}.

Table 2. (A). Defined order for tiles and their learned weights. (B). Sorted weights in the learned parameters $\theta_{1,fan}$ and their corresponding **fans**. Learned weights for $\theta_{2,fan}$ are added for comparison. (C). The **major fans** of ψ_2 and ψ_3 with a difference of at least 1% in historical frequency, which is calculated from their game history data.

A	Tile Name	Defined Order	$\theta_{1,tile}$
	White Dragon	1	1.41
	Green Dragon	2	1.13
	Red Dragon	3	1.07
B	**Fan** Name	$\theta_{1,fan}$	$\theta_{2,fan}$
	Seven Pairs	56.19	3.59
	Pure Terminal Chows	7.42	1.99
	Four Pure Shift Pungs	4.10	0.42
C	**Fan** Name	Frequency Diff $\psi_2 - \psi_3$ (%)	Param Diff $\theta_{2,fan} - \theta_{3,fan}$
	Pure Straight	1.30	0.45
	Mixed Straight	1.94	0.65
	Mixed Triple Chow	2.09	0.63
	Half Flush	1.04	0.27
	Mixed Shifted Chows	5.08	0.31
	All Types	2.60	0.27
	Melded Hand	−8.25	−1.24
	Fully Concealed Hand	1.83	−0.55

5.1.2. Preference of Tiles to Discard

Θ_{tile} stands for the relative preference of discarding each tile, especially when multiple **redundant tiles** are valued similarly. Since ψ_2 and ψ_3 show no apparent preference in discarding tiles, we focus on the analysis of ψ_1, which is constructed with a fixed tile preference. We find the learned weights almost form a monotonic sequence with only 3 exceptions out of 34 tiles, showing strong correlation between the learned parameters and the tile preferences of the target agent. Table 2A shows the first a few entries of $\theta_{1,tile}$.

5.2. Manipulation of Behaviors by Parameters

Previous experiments have shown that greater frequencies of **fans** within the game's historical record lead to elevated preferences. In this experiment, we illustrate that through the artificial augmentation of **fan** preferences, the modified agents, in contrast, display elevated frequencies of the corresponding **fans**. We make adjustments to the parameters $\theta_{2,fan}$ within $\hat{\psi}_2$ by multiplying the weight assigned to the *All Types* fan by a factor of 10 (The factor of 10 is an arbitrary but significant multiplier chosen to induce a strong and clearly observable shift in the agent's behavior, thereby demonstrating that the learned parameters in Θ_{fan} are actionable and directly influence strategic preferences.). This gives rise to the generation of a new agent, ψ_2', which is expected to exhibit a stronger inclination towards selecting this **fan**.

We collect roughly around 8000 self-play game data samples from ψ_2 and ψ_2'. Subsequently, we determine the frequency with which each fan shows up in their winning hands. The data indicates that among all the fans, only the *All Types* fan undergoes a frequency variation that surpasses 1%. Its frequency has risen from 2.59% to 5.76%, signifying an increment of 3.17%. In contrast, the frequencies of all other **major fans** have experienced changes of less than 1%. Given that Mahjong is a game characterized by a high level of randomness

in both the tile-dealing and tile-drawing processes, adjusting the parameters related to fan preferences can only bring about a reasonable alteration in the actual behavior patterns of the target agents. In conjunction with the findings of previous experiments, we can draw the conclusion that the parameters within Mxplainer are, in fact, significantly correlated with the behaviors of agents. Moreover, through the analysis of these parameters, we are capable of discerning the agents' preferences and high-level behavioral characteristics.

5.3. Interpretation of Deduction Process

In this subsection, we analyze the deduction process of the search-based framework F on an example game state by tracing the intermediate results of $\hat{\psi}_2$ to demonstrate the local explainability of Mxplainer agents in decision-making.

The selected game state S is at the beginning of a game with no tiles discarded yet, and the player needs to choose one to discard, as shown in Table 3A. The target black-box agent ψ_2, selects the tile B9 as the optimal choice to discard for unknown reasons. However, analyses of the execution of the white-box agent $\hat{\psi}_2$ explain the choice.

Table 3. (A). An example game state at the beginning of the game where no tiles have been discarded by other players. (B). Some proposed goals for state s by $\hat{\psi}_2$ and the estimated win probabilities. "H&K" is an abbreviation for *Honors and Knitted* due to space constraints.

A		Other Information			Current Hand
		None			
B	**ID**	**Major Fan**	Probs	**Redundant Tiles** R	Final Target
	25	*Lesser H&K Tiles* [1]	1.000	D7, C9, B6, B8, B9	
	0	*Knitted Straight* [1]	0.125	C9, B9, NW, RD, WD	
	11	*Pure Straight*	0.018	D2, C3, C6, NW, RD, WD	
	50	Seven Pairs [1]	0.017	B6, B8, B9, NW, RD, WD	

[1] A **fan** that does not follow the pattern of four melds and a pair.

The Search Component SC proposed 64 possible goals for s, and Table 3B shows a few goals representative of different **major fans**. With fitted θ_2, Algorithm 1 produces an estimated value for each goal G, as listed under the "Value" column of Table 3B. We observe that though B9 is required in goals such as *Pure Straight*, it is not required for many goals with higher estimated values, such as *Knitted Straight*.

Following Algorithm 2, we accumulate the values for tiles in **Redundant Tiles** R. The higher value of a tile indicates a higher value if the tile is discarded, and B9 turns out to have a higher value than other tiles by a large margin, which is consistent with the observed decision of ψ_2.

This section merely demonstrates an analysis of action selection within the context of a simple Mahjong state. However, such analyses can be readily extended to other complex states. The Search Component SC consistently takes in information and puts forward the top 64 reachable goals, ranked according to distances. The Calculation Component CC calculates the value for each goal using the learned parameters. Finally, an action is chosen based on these values.

Without Mxplainer, people can only observe the actions of black-box MahJong agents, but they cannot understand how the decisions are made. Our experiments show that Mxplainer's fitted parameters can well approximate and mimic target agents' behaviors. By examining the fitted parameters and Mxplainer's calculation processes, experts are able to interpret the considerations of black-box agents leading to their actions.

6. Ablation and Comparative Study

6.1. Number of Searched Goals

To assess potential bias from the 64-goal cap, we performed a sensitivity analysis on goal recall for the target agent ψ_2. Given that the probability of success decreases exponentially with Shanten Distance D (approximately by a factor of $1/34$ per unit increase), we specifically measured the recall rate for goals with the minimum D in each state.

A survey of approximately 10,000 game logs revealed that the average number of such minimum-D goals was 13.28. The coverage rate for these goals and the corresponding search time are summarized in Table 4.

We further evaluated the predictive accuracy of the approximated agent $\hat{\psi}_2$ under different goal caps (16, 32, 64, and 128). The results, illustrated in Figure 5, show that both Top-1 and Top-3 accuracy improve substantially when the cap is increased from 16 to 64. However, increasing the cap further to 128 yields only marginal gains, indicating diminishing returns beyond 64 goals.

Table 4. Recall rate and time cost for goals with different caps.

Cap (K)	Coverage Rate@K	Time (s)
16	79.31%	0.097
32	87.85%	0.098
64	96.82%	0.109
128	99.79%	0.128

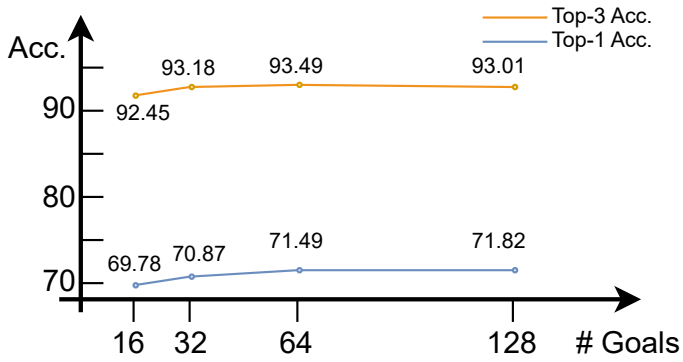

Figure 5. Top-1 and top-3 accuracy of $\hat{\psi}_2$ when the number of goals is 16/32/64/128.

6.2. Distribution Modeling

To rigorously evaluate the impact of probability model specification, we conducted an ablation study comparing different configurations for modeling p_{draw} and $p_{discard}$.

The results are summarized in Table 5. Here, "Network" denotes a learnable module that uses local game features Z to estimate the acquisition probability, whereas "Uniform" assumes a simple uniform distribution over unshown tiles. The configuration highlighted in bold was selected for our final framework, as it achieved the highest overall accuracy without a substantial increase in parameter count. This suggests that accurately modeling the draw probability from the wall is more critical for policy imitation than refining the discard model under the current framework.

Table 5. Ablation study on probability models for tile acquisition.

p_{draw}	$p_{discard}$	Top-1 Acc	Top-3 Acc	Parameter Size
Uniform	Uniform	66.26	90.65	114
Network	**Uniform**	**71.49**	**93.47**	**126**
Uniform	Network	67.35	91.16	126
Network	Network	71.49	93.44	138

6.3. Parameter Size and Methods of Behavior Cloning

In an effort to boost the explainability of Mxplainer, we have minimized the parameter size of the Small Network N. Nevertheless, this design might compromise its expressiveness and decrease the accuracy of the approximated agents. Consequently, we aim to further investigate the impact of the parameter size of the Small Network N on the approximation of the Target Agent Ψ. Specifically, we augment the parameter size and train networks $\hat{\psi}_2'$ and $\hat{\psi}_2''$ to approximate the identical Target Agent ψ_2. We do not elaborate on the modifications to the network structures herein. The parameter sizes and their respective top-3 accuracies are presented in Table 6A.

We observe that by increasing the parameter size, Mxplainer can approximate the Target Agent Ψ with greater accuracy. Nevertheless, the drawback lies in the fact that the larger the number of parameters in the Small Network N, the more challenging it becomes to explain the actions and the meaning of the parameters to users. Although the approximated agents cannot replicate the exact actions of the original black-box agents, they can account for their actions in most scenarios, as demonstrated by previous experiments.

We also conduct a comparison of the effects of different methods. Specifically, we construct a random forest consisting of 100 decision trees and employ a pure neural network to learn the behavior policy of ψ_2 through behavior cloning. The results are presented in Table 6B. Although decision trees are inherently self-explanatory, their low accuracies render them unsuitable for Mahjong, which involves numerous OOD states. The accuracy of Mxplainer is not notably lower than that of traditional neural networks with sufficient expressivity, but it has the advantage of being able to explain the reasoning underlying the actions.

Table 6. (A). Comparison between parameter size and resulting accuracies. (B). Comparison between different methods and accuracies.

A	Network	Param	θ_{fan}	θ_{held}	θ_{tile}	Top-3 Acc%
	$\hat{\psi}_2$	126	80	12	34	93.47
	$\hat{\psi}_2'$	2804	2720	50	34	93.77
	$\hat{\psi}_2''$	3280	2800	435	45	94.57
B	Method	Param		Acc%		Top-3 Acc%
	Mxplainer	126		71.49		93.47
	Decision Tree	N/A		28.91		34.82
	Behavior Clone	34M		75.31		95.69

7. Discussion

A primary limitation of Mxplainer is that its fidelity is inherently bounded by the expressiveness of the hand-crafted search framework F. Strategies that operate outside the paradigm of goal-search and linear parameterization may not be fully captured.

7.1. Choice of Probability Distributions for Tiles

A key design choice in the Mxplainer framework is the use of different probability models for drawing a tile from the wall (p_{draw}) and for an opponent discarding a tile ($p_{discard}$). We model p_{draw} with a learnable neural component that incorporates local game features, while $p_{discard}$ is modeled using a uniform distribution over the unshown tiles.

This choice reflects a deliberate trade-off between expressiveness and interpretability. Modeling p_{draw} with a learnable network is both feasible and critical. The wall is a passive, stochastic element; its state can be reasonably approximated by tracking unshown tiles. A model that learns to weight features like total unshown tiles, game length, and availability of adjacent tiles (Z) can effectively capture the nuanced likelihood of drawing a specific tile, providing a significant accuracy boost over a uniform assumption, as shown in our ablation study (Section 6.2).

In contrast, we employ a uniform distribution for $p_{discard}$. This assumption is a significant simplification, as it likely introduces a systematic bias by overstating the melding probability for tiles that opponents would strategically withhold. In theory, this could inflate the learned value of goals that rely on melding such tiles.

However, our ablation study (Section 6.2) demonstrates that the core conclusions of Mxplainer, specifically, its high imitation accuracy and the strategic insights derived from Θ, are remarkably robust to this simplification. The minimal performance gap between uniform and learned discard models suggests that the framework successfully absorbs much of this potential bias into the other calibrated parameters (e.g., Θ_{fan}) while maintaining a highly interpretable structure. Therefore, while a more sophisticated opponent model is a worthwhile goal for future work, the uniform assumption is not a prerequisite for Mxplainer's current ability to deliver faithful and explainable agent imitations. We prioritized this simpler, interpretable model to maintain a clear causal chain of reasoning, accepting a theoretically sub-optimal but empirically adequate approximation for the purpose of high-fidelity policy imitation.

Furthermore, the imitation accuracy of Mxplainer approaches that of a large, black-box neural network trained via behavior cloning (see Table 6B). This near-state-of-the-art performance, achieved with only 126 interpretable parameters compared to 34 million, underscores the efficiency of our parameterized search framework. It validates that the high fidelity of the approximation is not sacrificed for explainability but rather achieved through a structured, interpretable model.

7.2. Compare Characteristics of Agents with Mxplainer

With Mxplainer, we can compare the differences between agents. By comparing the fitted parameters in parallel with other agents, we can analyze the characteristics of different agents. For example, we can easily observe that the black-box AI agent ψ_2 has a much higher weight on *Thirteen Orphans*, *Seven Pairs*, and *Lesser H&K Tiles*. In contrast, the human player ψ_3 has a significant inclination of making *Melded Hand*, and these observations are indeed backed by their historical wins in Supplemental Material Table S2. Note that while Mxplainer successfully profiles an individual's strategy, its ability to capture population-level human playstyles requires future study with a broader dataset.

7.3. Limitations

While Mxplainer provides faithful and interpretable approximations, its fidelity is inherently bounded by the expressiveness of the manually constructed search-based framework F. Strategies that operate on principles fundamentally beyond the template's goal-search logic and linear parameterization—for instance, those relying on complex, non-linear state evaluations or long-term deceptive plays not captured by our goal-oriented

model—may not be fully captured. This is a deliberate trade-off we make to prioritize interpretability, as the components of F are designed to be comprehensible to domain experts.

7.4. Extend Mxplainer to Other Settings and Future Direction

Our proposed approach has a unique advantage in quantifying and tuning weights for custom-defined task-related features in areas where interpretability and performance are crucial. While Mxplainer is specifically designed for Mahjong, we hypothesize that its unique approach and XAI techniques may be applicable to other applications. In fact, we experimentally applied our proposed framework to two examples: Mountain Car from Gym [26] and Blackjack [34]. Both examples, which can be found in Appendix A, confirmed the effectiveness of our proposed method. However, the scope of application of our method still requires further study.

Another compelling future direction is to extend Mxplainer into a multimodal framework. By incorporating auxiliary data streams—such as eye-tracking to measure attention, heart rate variability to assess cognitive load or stress, and mouse-movement dynamics—we could move beyond modeling a static strategic policy. This would allow us to create a context-aware model that explains how a player's decisions are modulated by their real-time psychological and physiological state. Such an expansion could bridge the gap between pure game-theoretic strategy and the rich, situated reality of human decision-making.

8. Conclusions

In this work, we introduced Mxplainer, a novel framework that bridges the gap between the high performance of black-box agents and the critical need for interpretability in complex domains like Mahjong. Unlike existing imitation learning or post hoc explanation methods that often trade accuracy for transparency or struggle with high-dimensional state spaces, Mxplainer's unique approach constructs a parameterized, search-based agent whose components are directly convertible into an equivalent, trainable neural network. This core design allows it to accurately approximate sophisticated agents while retaining a human-understandable deduction process and strategically meaningful parameters.

Our experiments demonstrate that Mxplainer successfully extracts and quantifies agent characteristics, such as fan and tile preferences, and provides local explanations for individual decisions. The framework's ability to manipulate agent behavior by tuning these parameters further validates that the learned values capture genuine strategic inclinations.

Nevertheless, Mxplainer has limitations. Its approximation power is inherently constrained by the expressiveness of the hand-crafted search framework, and the current use of uniform distributions for tile-drawing and discarding probabilities is a simplifying assumption that could be refined. Future work will focus on enhancing the model's expressiveness without sacrificing interpretability, exploring more sophisticated probability models, and, most importantly, generalizing the explanation template to a wider range of applications beyond Mahjong. We believe that the paradigm of converting a domain-specific, parameterized classical agent into a trainable network offers a promising path forward for building performant, transparent, and trustworthy AI systems.

Supplementary Materials: The following supporting information can be downloaded at https://www.mdpi.com/article/10.3390/a18120738/s1. Table S1: Correlation Between Learned Parameters and Tile Preference; Table S2: Correlation Between Learned Parameters and Fan Preference; Table S3: Interpretation of Deduction Process For Sampled Game State.

Author Contributions: Conceptualization, L.L.; methodology, L.L.; software, L.L.; validation, L.L.; investigation, L.L., Y.L., Y.W.; data curation, L.L. and Y.L.; writing—original draft preparation, L.L.; writing—review and editing, L.L., Y.L., Y.W., W.L. and Q.Z.; visualization, L.L.; supervision, W.L.; project administration, W.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data presented in this study are openly available in Mxplainer at https://github.com/Lingfeng158/Mxplainer, accessed on 1 November 2025.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Mounter Car and Black Jack

In Mountain Car, the observation space is location **L** and speed **S**, both of which can be positive or negative. The action spaces are accelerations to the left or right. We can design a heuristic template H that takes the sign of **L** and **S** as input, and the heuristics, Θ, are actions for these four states. The transformation T is also simple: We map **L** and **S** to the four states during data processing and output a one-hot mask for Θ. When imitating the optimal strategy, the learned Θ can be found in Figure A1A, which also achieves optimal control.

In Blackjack, the observation space includes usable ace **A**, dealer's first card **C**, and player's sum of points **S**. The action space is to hit or stick under each state. The Heuristic Template H is designed to act based on heuristic boundaries Θ of **S** for Hit or Stick for each (**A**, **C**) pair. The transformation T is again mapping (**A**, **C**) pairs to their masks. The optimal strategy and the learned Θ is shown in Figure A1B, showing that the framework learns exactly the same behavior with the target model, i.e., the optimal strategy.

A.

	L<=0, S<=0	L<=0, S>0	L>0, S<=0	L>0, S>0
Optimal	0	2	0	2
Learned	0.0158	2.0039	-0.0034	2.0047

B.

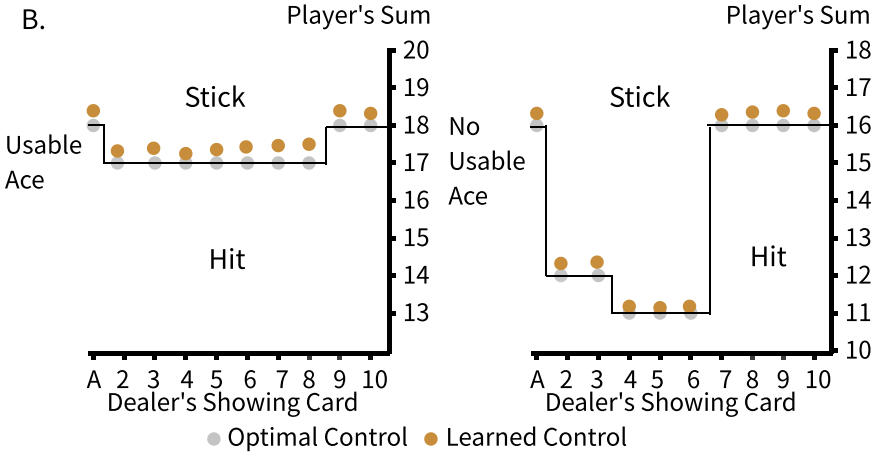

Figure A1. Optimal parameters and learned parameters for (**A**) mountain car and (**B**) black jack.

Appendix B. Rules and Examples of Transformation of Non-Differential Structures

Boolean-to-Arithmetic Masking (M) is defined as a one-hot vector of dim $|n|$ for a conditional statement with n branches. If a conditional branch can be determined without

parameters, M can be computed and stored as input data; otherwise, M can be computed at runtime. Computations for results R are the same for all branches, but only one result is activated through $R \cdot M$.

The *Padding Value* (P) is defined as the identity element of its related operator. Identity elements ensure that any computation paths with P produce no-op results, and they are required for the treatment of loops. For example, if the results of a loop are gathered through a summation operation, then the computation path with P produces 0. If the results are gathered through multiplication, then the path with P produces 1. This is because for classical agents to be optimized and parallelized, loops must be set with upper limits, similarly to the maximum sequence length when training a recurrent neural network. P needs to fill the iterations where there is no actual data. To keep the Framework F exactly the same as the transformed network N, all upper limits of loops need to be reflected in the original loops of F. Table A1 summarizes M and P.

Table A1. Examples of the transformation for conditional statements and loops. Note that classical representations of loops need to be modified with the same upper limit as the neural representations. $V^{\top[1,2]}$ is the example input with axis 1 and 2 transposed for efficient representation due to space constraints.

Conditional	Classical Representation	Neural Representation (Pytorch)

	`def foo(a,b):`
	`  if P:`
	`  r = a*3+b*4`
	`  return r`
	`  elif Q:`
	`  return a*b`
	`  else`
	`  return a/b`
		`def forward(A, B, M):`
		`  p = A*3 + B*4`
		`  q = A*B`
		`  o = A/B`
		`  out = torch.cat([p,q,o], dim=1)`
		`  return torch.sum(out*M, dim=1)`

Loop A	**Classical Representation**	**Neural Representation (Pytorch)**

Note: Results are independent between Iterations.	`def bar1(A):`
	`  res = []`
	`  e = enumerate`
	`  for i,a in e(A):`
	`  if i>64:`
	`  break`
	`  r = 2*(a+1)`
	`  res.append(r)`
	`  p = np.prod`
	`  return p(res)`
		`def forward(V):`
		`  V1 = V.view(n*64, 2)`
		`  O = V1[:,0]*2*(V1[:,1]+1)`
		`  # convert padding values to 1`
		`  # for the final torch.prod()`
		`  O += 1-V1[:, 0]`
		`  O1 = O.view(n, 64)`
		`  return torch.prod(O1, dim=1)`

Loop B	**Classical Representation**	**Neural Representation (Pytorch)**

Note: Results are dependent between Iterations.	`def bar2(A):`
	`  res = []`
	`  e = enumerate`
	`  for i,a in e(A):`
	`  if i>64:`
	`  break`
	`  r=a`
	`  if i!=0:`
	`  r += res[-1]`
	`  res.append(r)`
	`  return sum(res)`
		`def forward(V):`
		`  res = []`
		`  for i in range(64):`
		`  if i==0:`
		`  r = V[:, i, 1]`
		`  else:`
		`  # computing real data`
		`  r = res[-1]+V[:, i, 1]`
		`  # set zero for padding values`
		`  r*=V[:, i, 0]`
		`  res.append(r)`
		`  res = torch.stack(res)`
		`  res = torch.sum(res, dim=0)`
		`  return res`

We provide a lemma to show that M and P result in equivalent F and N.

Lemma A1. *Semantic Equivalence under Identity Padding and Masking*

Let F be a classical computation framework containing loops with a fixed upper limit L and conditional branches. Let N be its neural network transformation, implemented using the following:

- *A padding value P that is the identity element for the accumulation operation in a loop (e.g., 0 for summation, 1 for product).*
- *A Boolean mask M that is a one-hot vector for conditional branches, selecting the active computational path.*

If the following conditions hold, then for any input S, the output of $N(S)$ is identical to the output of $F(S)$:

- *All loops in F are padded to the same upper limit L as in N, with padding elements set to P.*
- *All branch conditions in F are deterministically mapped to their corresponding one-hot masks M in N.*
- *The accumulation operations (e.g., sum, prod) in F are associative and commutative, and P is their true identity element.*

Proof. Base Case

For elementary operations (e.g., arithmetic operations on tensors that do not involve control flow), the transformation T is the identity function. By definition, $N(S) = F(S)$ for these operations.

Conditional Branching

Consider a conditional statement in F with n branches.

Listing A1. Branching in Python

```
# Classical Representation F
if condition_1:
    result = f_1(S)
if condition_2:
    result = f_2(S)
...
if condition_n:
    result = f_n(S)
```

This is transformed in N into the following.

Listing A2. Branching in Neural Network

```
# Neural Representation N
M = one_hot(condition_1, condition_2) #Mask Vector
R = stack([f_1(S), f_2(S), ..., f_n(S)]) # All Branch Results
result = sum(R * M)  # Mask application
```

By Condition 2, the branch conditions are deterministically mapped to a one-hot mask M, where exactly one element is $M[i] = 1$, and all others are 0. The stack operation computes all branch results $f_1(S), f_2(S), ..., f_n(S)$ in parallel. The operation $sum(R * M)$ performs an element-wise multiplication of the result tensor R with the mask M. Since M is one-hot, this operation selects the i-th branch result $f_i(S)$ and sums it with zeros from all other branches. Therefore, the result in N is identical to the result in F, as both equal $f_i(S)$.

Fixed-Bound Loops

Consider a loop in F with an upper bound L, which accumulates results using an operator $\oplus$ with identity element P:

Listing A3. Fixed Bound Loops in Python

[mathescape=true]

```
# Classical Representation F
result = P                                  # Initialize with identity
for i in range(L):
    if i < actual_length:
        element = get_element(i, S)
        result = result ⊕ element
    # Implicit else: pad with identity~P
```

This is transformed in N into the following.

Listing A4. Fixed Bound Loops in Python

```
# Neural Representation N
all_elements = get_all_elements(S, L)   # Padded with P to length L
result = reduce(all_elements, ⊕)        # Parallel reduction with ⊕
```

By Condition 1, the loop in F is explicitly padded to length L with the identity element P. The neural version N directly operates on this padded sequence *all_elements*. By Condition 3, the accumulation operation $\oplus$ is associative and commutative. The fundamental property of an identity element P is that for any x, $x \oplus P = P \oplus x = x$. Therefore, performing the reduction $\oplus$ over the padded sequence *all_elements* is equivalent to performing the sequential loop in F: The contributions of the real elements are unchanged, and the padding elements P leave the accumulation result invariant. Thus, the result computed by N is identical to that computed by F. □

This lemma provides the formal basis for our transformation. The padding value P ensures that padded elements do not alter the result of the accumulation, while the mask M ensures that only the correct branch result is propagated.

To empirically verify the equivalence between framework F and network N under varied conditions, we conducted large-scale differential testing. We sampled 10,000 game states from our database and generated random parameter weights for N, which were then directly transferred to F. The outputs of both systems—specifically, the values computed by Algorithm 1/Listing 1 and the tile preferences computed by Algorithm 2/Listing 2—were compared for every state. The results were identical across all 10,000 test cases, confirming the semantic equivalence of F and N.

References

1. Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; et al. Mastering the game of Go without human knowledge. *Nature* **2017**, *550*, 354–359. [CrossRef]
2. Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv* **2017**, arXiv:1712.01815. [CrossRef]
3. Brown, N.; Sandholm, T. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science* **2018**, *359*, 418–424. [CrossRef] [PubMed]
4. Brown, N.; Sandholm, T. Superhuman AI for multiplayer poker. *Science* **2019**, *365*, 885–890. [CrossRef] [PubMed]
5. Yang, G.; Liu, M.; Hong, W.; Zhang, W.; Fang, F.; Zeng, G.; Lin, Y. PerfectDou: Dominating DouDizhu with Perfect Information Distillation. *arXiv* **2024**, arXiv:2203.16406. [CrossRef]
6. Zha, D.; Xie, J.; Ma, W.; Zhang, S.; Lian, X.; Hu, X.; Liu, J. DouZero: Mastering DouDizhu with Self-Play Deep Reinforcement Learning. *arXiv* **2021**, arXiv:2106.06135. [CrossRef]
7. Li, J.; Koyamada, S.; Ye, Q.; Liu, G.; Wang, C.; Yang, R.; Zhao, L.; Qin, T.; Liu, T.Y.; Hon, H.W. Suphx: Mastering Mahjong with Deep Reinforcement Learning. *arXiv* **2020**, arXiv:2003.13590. [CrossRef]

8. Vinyals, O.; Babuschkin, I.; Czarnecki, W.M.; Mathieu, M.; Dudzik, A.; Chung, J.; Choi, D.H.; Powell, R.; Ewalds, T.; Georgiev, P.; et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* **2019**, *575*, 350–354. [CrossRef]
9. Fan, L.; Wang, G.; Jiang, Y.; Mandlekar, A.; Yang, Y.; Zhu, H.; Tang, A.; Huang, D.A.; Zhu, Y.; Anandkumar, A. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. *arXiv* **2022**, arXiv:2206.08853. [CrossRef]
10. Wei, H.; Chen, J.; Ji, X.; Qin, H.; Deng, M.; Li, S.; Wang, L.; Zhang, W.; Yu, Y.; Liu, L.; et al. Honor of Kings Arena: An Environment for Generalization in Competitive Reinforcement Learning. *arXiv* **2022**, arXiv:2209.08483. . [CrossRef]
11. Silver, D.; Huang, A.; Maddison, C.J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **2016**, *529*, 484–489. [CrossRef] [PubMed]
12. Willingham, E. AI's Victories in Go Inspire Better Human Game Playing. 2023. Available online: https://www.scientificamerican.com/article/ais-victories-in-go-inspire-better-human-game-playing/ (accessed on 12 April 2024).
13. Huang, A.; Hui, F.; Baker, L. AlphaGo Teach. 2017. Available online: https://alphagoteach.deepmind.com/ (accessed on 12 April 2024).
14. Saedol, L. 8 Years Later: A World Go Champion's Reflections on AlphaGo. 2024. Available online: https://blog.google/around-the-globe/google-asia/8-years-later-a-world-go-champions-reflections-on-alphago/ (accessed on 12 April 2024).
15. Li, J.; Wu, S.; Fu, H.; Fu, Q.; Zhao, E.; Xing, J. Speedup Training Artificial Intelligence for Mahjong via Reward Variance Reduction. In Proceedings of the 2022 IEEE Conference on Games (CoG), Beijing, China, 21–24 August 2022; pp. 345–352. [CrossRef]
16. Zhao, X.; Holden, S.B. Building a 3-Player Mahjong AI using Deep Reinforcement Learning. *arXiv* **2022**, arXiv:2202.12847. [CrossRef]
17. Arrieta, A.B.; Díaz-Rodríguez, N.; Ser, J.D.; Bennetot, A.; Tabik, S.; Barbado, A.; García, S.; Gil-López, S.; Molina, D.; Benjamins, R.; et al. Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI. *arXiv* **2019**, arXiv:1910.10045. [CrossRef]
18. Linardatos, P.; Papastefanopoulos, V.; Kotsiantis, S. Explainable AI: A Review of Machine Learning Interpretability Methods. *Entropy* **2021**, *23*, 18. [CrossRef]
19. Ribeiro, M.T.; Singh, S.; Guestrin, C. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *arXiv* **2016**, arXiv:1602.04938. [CrossRef]
20. Liu, G.; Schulte, O.; Zhu, W.; Li, Q. Toward Interpretable Deep Reinforcement Learning with Linear Model U-Trees. In *Proceedings of the Machine Learning and Knowledge Discovery in Databases*; Berlingerio, M., Bonchi, F., Gärtner, T., Hurley, N., Ifrim, G., Eds.; Springer: Cham, Switzerland, 2019; pp. 414–429.
21. Milani, S.; Topin, N.; Veloso, M.; Fang, F. Explainable Reinforcement Learning: A Survey and Comparative Review. *ACM Comput. Surv.* **2024**, *56*, 168. [CrossRef]
22. Abbeel, P.; Ng, A.Y. Apprenticeship learning via inverse reinforcement learning. In Proceedings of the Twenty-First International Conference on Machine Learning, Banff, AB, Canada, 4–8 July 2004; ICML '04; p. 1. [CrossRef]
23. Schaal, S. Is imitation learning the route to humanoid robots? *Trends Cogn. Sci.* **1999**, *3*, 233–242. [CrossRef]
24. Jhunjhunwala, A.; Lee, J.; Sedwards, S.; Abdelzad, V.; Czarnecki, K. Improved Policy Extraction via Online Q-Value Distillation. In Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN), Glasgow, UK, 19–24 July 2020; pp. 1–8. [CrossRef]
25. Zhang, H.; Zhou, A.; Lin, X. Interpretable policy derivation for reinforcement learning based on evolutionary feature synthesis. *Complex Intell. Syst.* **2020**, *6*, 741–753. [CrossRef]
26. Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; Zaremba, W. OpenAI Gym. *arXiv* **2016**, arXiv:1606.01540. [CrossRef]
27. Verma, A.; Murali, V.; Singh, R.; Kohli, P.; Chaudhuri, S. Programmatically Interpretable Reinforcement Learning. In Proceedings of the 35th International Conference on Machine Learning, PMLR, Stockholm, Sweden, 10–15 July 2018; Dy, J., Krause, A., Eds.; Proceedings of Machine Learning Research; Volume 80, pp. 5045–5054.
28. Lu, Y.; Li, W.; Li, W. Official International Mahjong: A New Playground for AI Research. *Algorithms* **2023**, *16*, 235. [CrossRef]
29. Novikov, V. *Handbook on Mahjong Competition Rules*; European Mahjong Association: Copenhagen, Denmark, 2016.
30. Geißler, B.; Martin, A.; Morsi, A.; Schewe, L. Using Piecewise Linear Functions for Solving MINLPs. In *Proceedings of the Mixed Integer Nonlinear Programming*; Lee, J., Leyffer, S., Eds.; Springer: New York, NY, USA, 2012; pp. 287–314.
31. Griewank, A. On stable piecewise linearization and generalized algorithmic differentiation. *Optim. Methods Softw.* **2013**, *28*, 1139–1178. [CrossRef]
32. Morteza, A.; Chou, R.A. Constrained Optimization of Access Functions in Uniform Secret Sharing. In Proceedings of the 2025 IEEE International Symposium on Information Theory (ISIT), Ann Arbor, MI, USA, 22–27 June 2025; pp. 1–6. [CrossRef]

33. Zhou, H.; Zhang, H.; Zhou, Y.; Wang, X.; Li, W. Botzone: An Online Multi-Agent Competitive Platform for AI Education. In Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education, Larnaca, Cyprus, 2–4 July 2018; ITiCSE 2018; pp. 33–38. [CrossRef]
34. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; A Bradford Book; Cambridge University Press: Cambridge, MA, USA, 2018.